

CONFIDENTIAL — MATERA SYSTEMS (OR SUBSIDIARY)

This document is the exclusive intellectual property of Matera Systems or one of its subsidiaries ("Matera"). It is strictly confidential and intended solely for the authorized recipient(s) named herein.

Unauthorized use, reproduction, distribution, or disclosure of this document — in whole or in part — is strictly prohibited without prior written consent from Matera.

AI & SOFTWARE GENERATION NOTICE: This document, or any portion thereof, may not be used — directly or indirectly — as input, training data, prompt, or reference material for the development, generation, or derivation of any software, model, or system, whether through artificial intelligence tools or any other means. Any software, output, or derivative work produced in violation of this restriction shall be deemed the exclusive intellectual property of Matera, regardless of the party that produced it.

© Matera Systems. All rights reserved.

Table of Contents

Overview..... 3

Systems Architecture..... 3

Architecture..... 4

Development vs. Production.....4

Installation..... 5

 Step 1 – Create a Namespace..... 5

 Step 2 – Configure the Application.....5

 Step 3 – Deploy the Application.....7

 Step 4 – Validate the Deployment..... 8

Configuration Reference..... 8

Security Considerations..... 9

Additional Resources..... 9

Overview

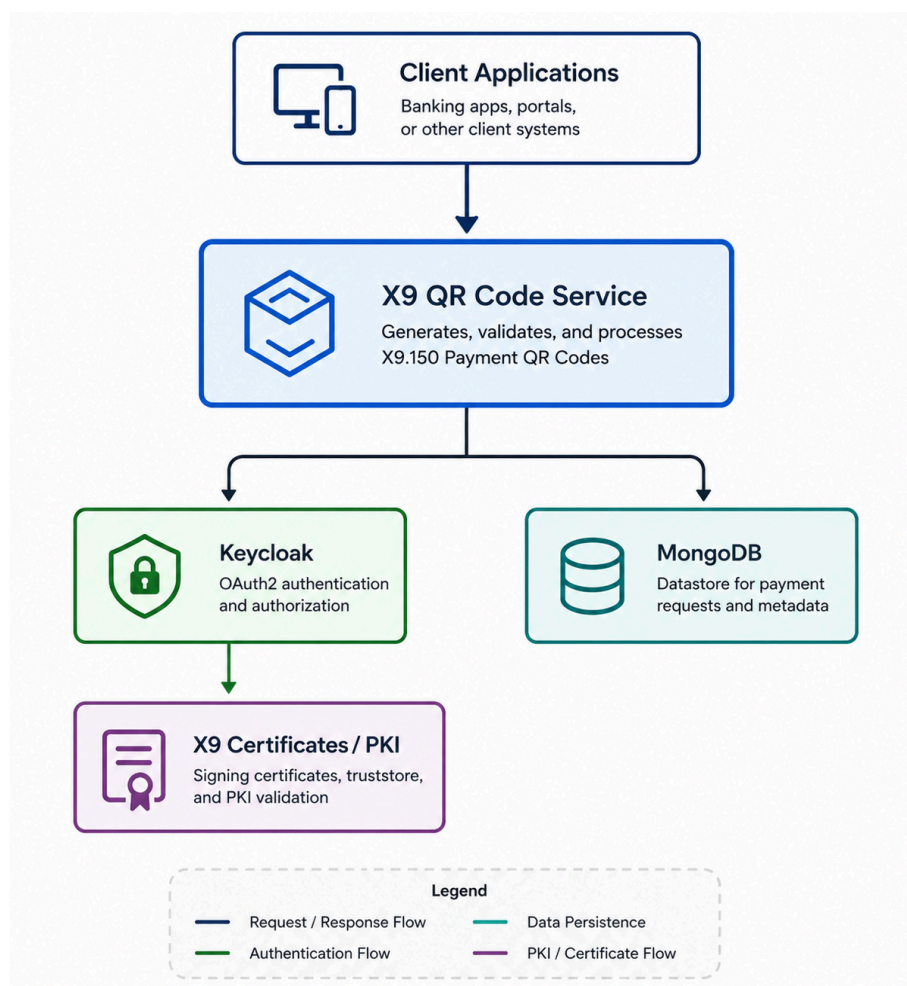
This guide explains how to deploy Madera's Payment QR Code software in a Kubernetes environment using Helm.

The software enables users to generate, validate, and process ANSI X9.150-compliant Payment QR Codes for account-to-account payment initiation.

It supports the complete payment request lifecycle, including Payment Payload and Payment Notification flows. It protects message integrity and authenticity using JSON Web Signatures (JWS) and X9 Financial PKI trust principles.

Systems Architecture

The Payment QR Code software integrates with several infrastructure components to provide authentication, persistence, and certificate management.



Architecture

The software is deployed as a containerized Java service in Kubernetes and integrates with standard infrastructure components for authentication, persistence, and certificate management.

- Java / Spring Boot application
- Kubernetes deployment using Helm
- MongoDB datastore (production)
- Keycloak for OAuth2 authentication
- X9 certificate management for signing and validation

Note: MySQL is supported only for development and testing. MongoDB is the recommended production datastore.

Before You Begin

Ensure the following components are available before installing the application.

Requirement	Purpose
Kubernetes cluster	Hosts the application
Helm 3.x	Deploys the application
Container registry access	Pulls the application image
MongoDB	Stores payment request data
Keycloak	OAuth2 authentication
X9 certificates	Signing and validation
Kubernetes namespace	Deployment target
Image pull secret	Registry authentication

Development vs. Production

The Helm chart supports both development and production deployments.

Development	Production
MySQL (optional)	MongoDB
Security may be disabled	OAuth2 authentication required
Demo certificates	Production certificates
Local or Kubernetes Secrets	External secret management recommended

The sample configuration in this guide is intended as a starting point for development environments. Production deployments should replace all sample credentials, certificates, and secrets.

Installation

Installing X9 QR Code consists of four steps:

1. Prepare the Kubernetes environment.
2. Configure the application.
3. Deploy the Helm chart.
4. Validate the installation.

Step 1 – Create a Namespace

Create (or select) the Kubernetes namespace where the application will be installed.

None

```
kubectl create namespace <namespace>
```

Step 2 – Configure the Application

The application requires several configuration settings before it can start successfully.

Container Image

None

```
image:
  repository: registry.company-domain.com/fednow/x9-qrcode
```

Authentication (Keycloak)

Configure the OAuth2 provider used to secure the application.

None

```
application:
  keycloak:
    host: https://keycloak.company-domain.com
    realm: Matera
```

Public Endpoint

Specify the public URL used in X9 payment payloads.

None

```
application:
  x9:
    public-endpoints:
      host: https://x9.company-domain.com
```

Certificates

Configure the X9 signing certificate and truststore.

None

```
application:
  x9:
    certificate:
      private-keystore:
        location: /certificate/x9-matera.jks
    truststore:
      location: /certificate/x9-truststore.jks
```

Secrets

Sensitive values should be stored separately from the application configuration.

Typical secrets include:

- MongoDB connection string
- Keycloak client secret
- Keystore password
- Private key password
- Truststore password

Example:

```
None
secrets:
  spring:
    data:
      mongodb:
        uri: mongodb+srv://...
```

Step 3 – Deploy the Application

Install or upgrade the Helm release.

```
None
helm upgrade --install x9-qr code <repo-name>/x9-qr code \
  --namespace <namespace> \
  --values values.yaml
```

To validate the generated Kubernetes manifests before deployment:

```
None
helm template x9-qr code <repo-name>/x9-qr code \
  --values values.yaml
```

Step 4 – Validate the Deployment

After deployment, verify that the application has started successfully and is ready to accept requests.

- Pods are running
- Readiness probes succeed
- Liveness probes succeed
- MongoDB connectivity is established
- Keycloak authentication is working

Confirm that the following API groups are available:

- Payment Request
- Payment Notification
- Signature Services
- Public X9 Endpoints

Complete API details are available in the project's OpenAPI specification.

Configuration Reference

The Helm chart provides configuration options for:

- Naming
- Images
- Service Accounts
- Deployment Strategy
- Runtime Environment
- Networking
- Autoscaling
- Resource Limits
- Health Probes
- Scheduling
- Application Configuration
- Secrets

The complete default configuration can be exported using:

None

```
helm show values <repo-name>/x9-qrcode > values-full.yaml
```

Security Considerations

For production deployments:

- Enable OAuth2 authentication.
- Use MongoDB with TLS enabled.
- Store secrets in a secure secret management solution.
- Replace all sample certificates.
- Replace all sample passwords.
- Validate certificate trust using the X9 Financial PKI.

Additional Resources

For complete configuration details, refer to:

- Helm Values Reference
- OpenAPI Specification
- Matera Portal Documentation